1 Lecture 17 (Oct 22) - Graph Layering, Whatever-First Search Variants, Depth-First Search

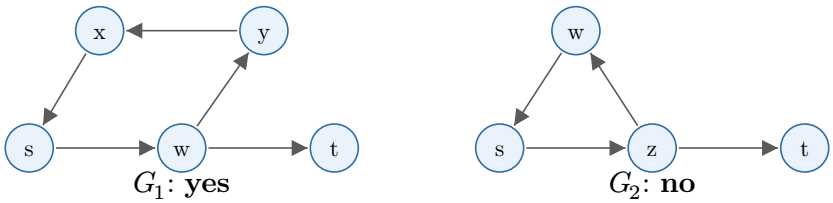
Today: one more reduction to reachability, with a technique called **graph layering**. Then what happens when the bag in whatever-first search matters, and a closer look at **depth-first search**.

1.1 Graph Layering

1.1.1 The problem

Input: a directed graph $G = (V, E)$ and two vertices s and t. **Question:** is there a **walk** from s to t whose length is divisible by 3?

A **walk** follows the edge directions, and may repeat vertices and edges. (A path may not.)



G_1: **yes** G_2: **no**

- G_1: the shortest path $s \to w \to t$ has length 2. But the walk $s \to w \to y \to x \to s \to w \to t$ has length 6. Every walk from s to t has length $2 + 4k$, and $k = 1$ works.
- G_2: there are infinitely many walks from s to t, such as $s \to z \to t$ and $s \to z \to w \to s \to z \to t$. But every one has length $2 + 3k$, which is never divisible by 3.

1.1.2 Don't modify the search, build a new graph

It's tempting to change the code of whatever-first search so it also tracks lengths mod 3. That can work, but it's **dangerous**. It's easy to get wrong, and you would have to prove that your modified search still finds every walk. Instead, **reduce to plain reachability** in a new graph G', and run the search unchanged.

Imagine an ant walking on G. To answer the question, it only needs to know two things: **where it is**, and **how many steps it has taken, mod 3**. Make each pair a vertex.

1. Build a graph: $G' = (V', E')$ with

$$V' = V \times \{0, 1, 2\}$$
$$E' = \{(u, i) \to (v, (i + 1) \bmod 3) \mid u \to v \in E, i \in \{0, 1, 2\}\}$$

Each edge of G becomes 3 edges of G', one **from each layer to the next**. Following an edge always adds one step. The arrow notation keeps this readable: an edge between pairs written with nested pairs gets ugly fast.

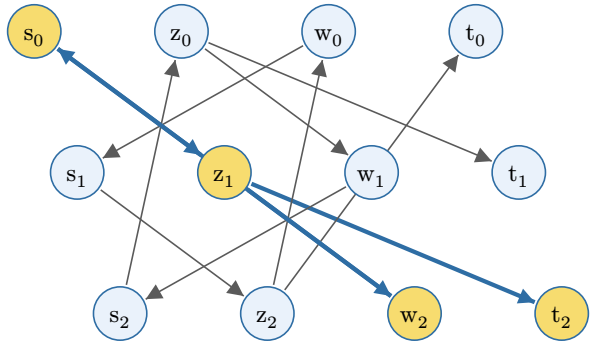
2. Graph problem, and why it helps:

G has a walk from s to t of length divisible by $3 \iff (s, 0)$ can reach $(t, 0)$ in G'

3. Algorithm: Run whatever-first search in G' from $(s, 0)$. Answer yes if and only if it marks $(t, 0)$.

4. Running time: $|V'| = 3|V|$ and $|E'| = 3|E|$, so building G' and searching it takes $O(|V'| + |E'|) = \boxed{O(|V| + |E|)}$. That's as fast as possible.

G_2, layered: Layer i holds the vertices v_i (meaning "at v, after i steps mod 3"). The search from s_0 marks the yellow vertices and follows the bold edges. It never marks t_0, so the answer is no:



Layers 0, 1, 2 from top to bottom. Every edge goes down one layer, except edges out of layer 2, which wrap around to layer 0.

For G_1, the walk of length 6 becomes the path $s_0 \to w_1 \to y_2 \to x_0 \to s_1 \to w_2 \to t_0$ in G'.

> **Key idea** **Graph layering:** when a graph problem (reachability, shortest paths) needs to remember **extra information**, make one copy of the graph per value of that information, and wire the copies together so the layer you're in **is** the information. Same idea as the product construction for DFAs: new states = old states × extra bits. Erickson has a separate note on graph layering, linked from the course schedule.

The new graph is an ordinary graph, so you can hand it to **any** black-box graph algorithm (whatever-first search, Dijkstra's algorithm, …). The algorithm doesn't care where the vertices came from.

1.2 Whatever-First Search, Revisited

1.2.1 Parent pointers

Put **pairs** into the bag instead of single vertices. A pair (p, v) means "found v, and the last step to it came from p":

```
WhateverFirstSearch(s):
  put (ø, s) into the bag
  while the bag is not empty
    take (p, v) from the bag
    if v is unmarked
```

```
mark v
v.parent ← p
for each edge v → w
  put (v, w) into the bag
```

Every marked vertex except s gets exactly one **parent edge** $v.\text{parent} \to v$, and its parent was marked earlier. So the parent edges form a **tree rooted at s**: it holds one path from s to every vertex s can reach. In an undirected graph, this is a **spanning tree** of s's component: a tree containing every vertex of it.

1.2.2 The bag matters now

A different bag removes vertices in a different order. That gives a **different tree**, and a different running time $O(|V| + |E| \cdot T)$, where T is the time per bag operation:

Bag	**Search, and the tree it builds**	T	**Total time**
Queue	**Breadth-first search**. Finds every vertex 1 step from s, then 2 steps, and so on. Builds a **shortest-path tree**, counting edges (no weights).	$O(1)$	$O(\|V\| + \|E\|)$
Priority queue: priority of (u, v) is $\text{dist}(u) + w(u \to v)$; weights ≥ 0	**Best-first search = Dijkstra's algorithm**. Shortest paths with edge weights (proof next week).	$O(\log\|E\|)$	$O(\|V\| + \|E\| \log\|V\|)$
Priority queue: priority of uv is $w(uv)$; undirected	**Prim's algorithm**. Builds a **minimum spanning tree** (not covered this semester).	$O(\log\|E\|)$	$O(\|V\| + \|E\| \log\|V\|)$
Stack	**Depth-first search**. Long, skinny trees. Uses: below.	$O(1)$	$O(\|V\| + \|E\|)$

Here $\text{dist}(u)$ is the total weight of the tree path from s to u. The priority queue is a binary heap, with $O(\log|E|)$ insert and extract-min. A simple graph has $|E| \leq |V|^2$, so $\log|E| \leq 2\log|V|$, and $O(\log|E|) = O(\log|V|)$.

> **Key idea** **BFS, DFS, Dijkstra and Prim are all whatever-first search:** They differ only in the bag that decides where to go next.

On homework 8 you may use Dijkstra's algorithm as a black box.

1.3 Depth-First Search

For BFS, Dijkstra and Prim there is an obvious problem to solve. For depth-first search there isn't a one-sentence reason to care. So this time, start with the algorithm and then look for problems it solves.

1.3.1 The recursive version

Whatever-first search with a stack is a depth-first search, but DFS is usually written **recursively**: for each edge $v \to w$, immediately explore everything out of w. One more change: check whether w is marked **before** the recursive call, instead of pushing it and checking later.

```
DFS(v):                        ⟨⟨v is unmarked⟩⟩
  mark v
  for each edge v → w
    if w is unmarked
      w.parent ← v
      DFS(w)
```

Like any search, DFS(s) marks everything reachable from s (if nothing was marked before). What that is depends heavily on s. On a path, starting at the first vertex marks everything, and starting at the last vertex marks only that vertex.

The uses of DFS are about the structure of the **whole** graph, so call it until every vertex is marked:

```
DFSAll(G):
  for all vertices v
    unmark v
  for all vertices v
    if v is unmarked
      DFS(v)
```

Every vertex is marked once and its edges are explored once: $O(|V| + |E|)$ time.

1.3.2 Three facts about DFSAll

1. **Every vertex gets marked**: DFS(v) is called exactly once for every v. The loop in DFSAll guarantees it.
2. **Before DFS(v) finishes, it marks every vertex reachable from v through unmarked vertices.** It keeps diving deeper. It stops only at vertices that someone else already marked.
3. **If DFS(v) calls DFS(w), directly or indirectly, there is a path from v to w.**

1.3.3 Clock: start and finish times

The applications depend on the **order** in which DFS starts and finishes each vertex. To reason about it, add a clock. Each event gets the next clock value:

```
DFSAll(G):
  clock ← 0
  for all vertices v
```

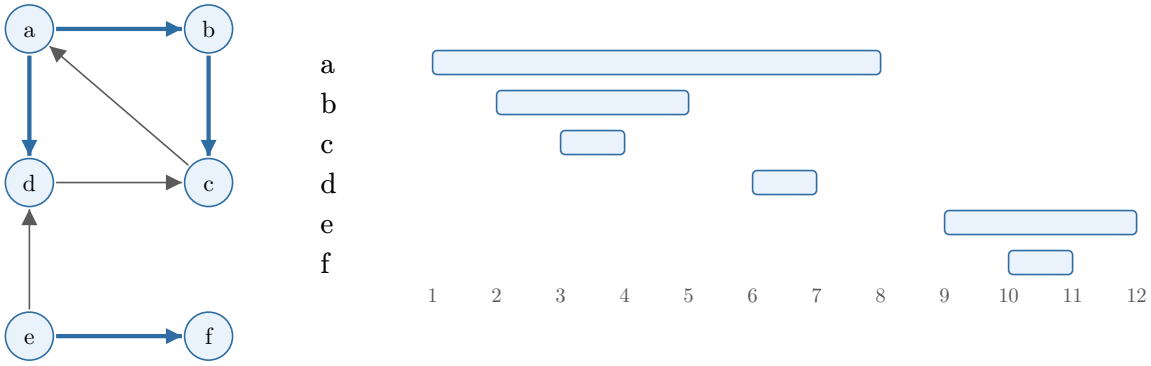
```
    unmark v
  for all vertices v
    if v is unmarked
      clock ← DFS(v, clock)

DFS(v, clock):
  mark v
  clock ← clock + 1
  v.pre ← clock                ⟨⟨start time⟩⟩
  for each edge v → w
    if w is unmarked
      w.parent ← v
      clock ← DFS(w, clock)
  clock ← clock + 1
  v.post ← clock               ⟨⟨finish time⟩⟩
  return clock
```

The **active interval** of v is $[v.\text{pre}, v.\text{post}]$: from when DFS($v$) goes onto the call stack until it comes off.

> **Key idea** **Active intervals are either disjoint or nested:** That is how a call stack works: if DFS(u) starts while DFS(v) is running, DFS(u) finishes first.

Example: DFSAll on this graph, taking vertices and edges in alphabetical order. Bold edges are tree (parent) edges:



DFS(a) finds b, then c (whose edge to a goes to a marked vertex), then backs up to a and finds d ($d \to c$: already marked). Nothing marked so far reaches e, so DFSAll calls DFS(e) later, which finds f.

Order	**Sort by**
Preorder: a, b, c, d, e, f	start time v.pre. Rarely useful in this class.
Postorder: c, b, d, a, f, e	finish time v.post. Very useful.

1.4 Finding a Cycle

A **dependency graph** has the subproblems of a recursive algorithm as vertices, and an edge $u \to v$ when u calls v directly (Fibonacci: $F_8 \to F_7$ and $F_8 \to F_6$). A recursive algorithm **terminates only if its dependency graph has no cycle**. On a cycle through u, solving u needs the answer to u, which you're still computing.

Problem: given a directed graph G, does it have a cycle? A directed graph with no cycle is a **directed acyclic graph** (DAG).

Run DFSAll. Suppose that during DFS(v) there is a call to DFS(u), and it finds an edge $u \to v$ back to the still-active v. That edge is a **back edge**. By fact 3 there is a path $v \rightsquigarrow u$, and with the edge $u \to v$ that makes a **cycle**. In the example, $c \to a$ is a back edge: cycle $a \to b \to c \to a$.

> **Key idea** **One back edge proves a cycle:** Which edges are back edges depends on the order DFSAll takes vertices and edges. But if DFSAll finds even one, the graph is not acyclic.

Is the converse true (a cycle $\Rightarrow$ DFS always finds a back edge)? Next lecture.